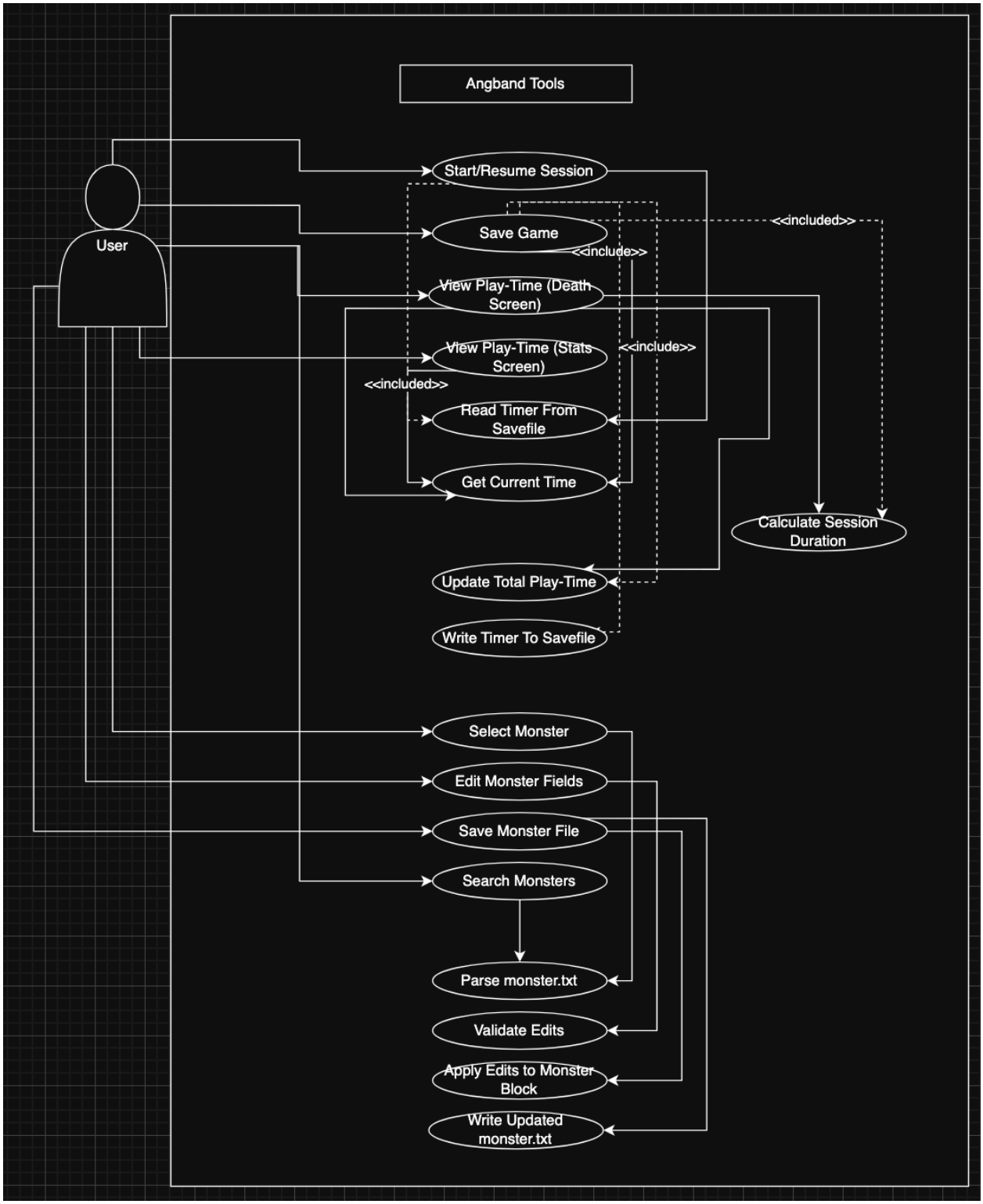
1. **Estimation Accuracy:** Analyze your actual time spent vs. your estimates from Part 1. Which task had the biggest discrepancy? What does this tell you about the difficulty of estimating software tasks?

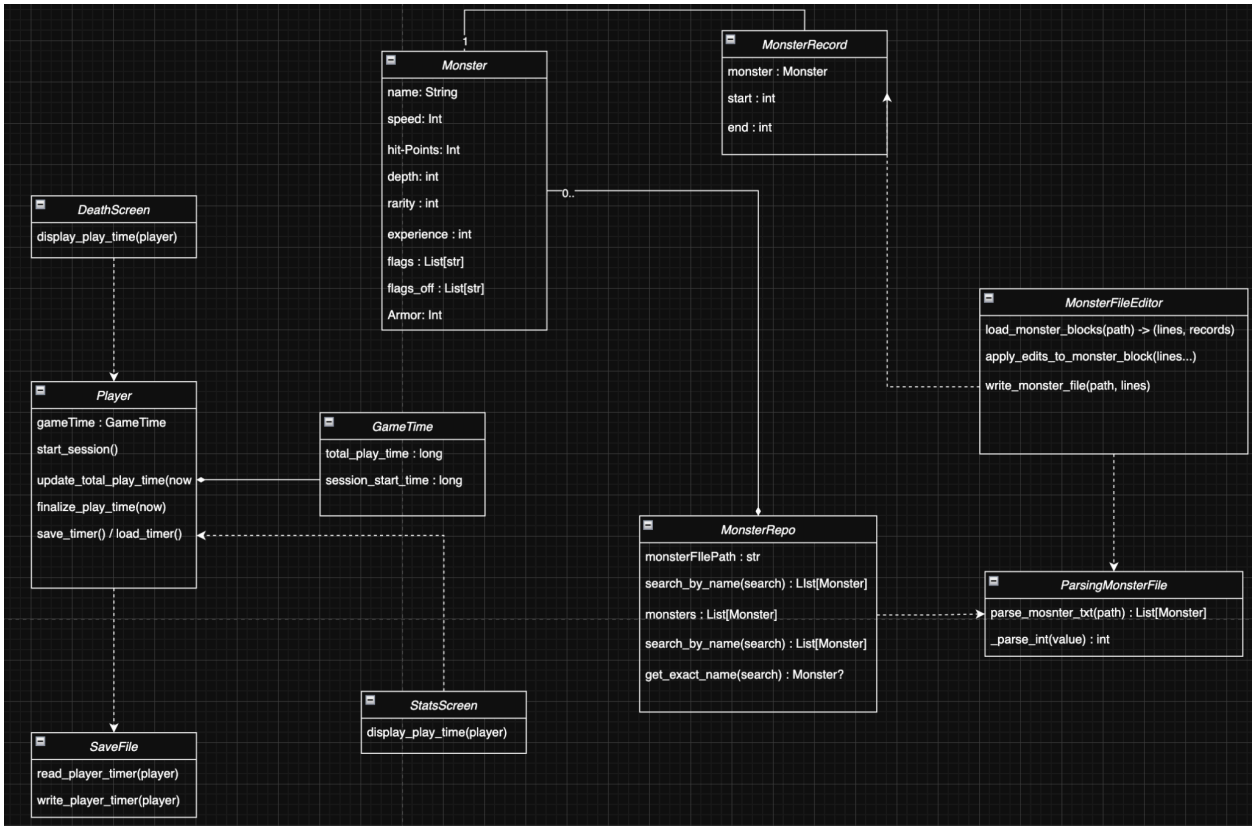
Angbang Time Analysis			
Project: Angband Timer and monster search implementation		Date: 02/25/2026	
Activity	Assignments: (can be adjusted as needed)	Time, estimated (in hours)	Time, actual (in hours)
Parsing Files For information	Elyse Emily	1	2.5
Data Allocation in C and its management- plan	Sergio	2	2
Classification logic and grouping monsters by dungeon level. Navigation and group by levels- Options for (1. Levels 0-10), (2. Levels 11-20),..., (0. Levels 91-100) ETC..	Chris Connor	3.5	3
Monster	Elyse	5.5	1.5

search by name			
Data Update for edited monster	Emily	3.5	9
Creation of timer for start and stop, a clear variable that is accessible to screen updates	Sergio Chris	3.5	4
Data Updating- for game character stats, both death and game character screens	Sergio	3.5	4
Testing- Trouble shooting, attempting to break code, Providing feedback to Correct people for fixes or assistance with fixes	Emmy connor	3	2
Instructions for running game document	Connor	1.5	1.5
[README.md](#) file with	Connor	1.5	1

instructions for installing game,and example screenshots			
Account of time spent	Elyse	1.5	2
Man page of program- https://www.cyberciti.biz/faq/linux-unix-creating-a-manpage/	Connor	2.5	1.5

Summary of time per person						
Person	Elyse	Emily	Sergio	Chris	Connor	Total together
Time per person (estimate)	6 hours	13.5 hours	10 hours	7 hours	9 hours	55 hours individual (counts 2 people working together as 2 hours) Collaborative- 27 (counts 2 people working together as 1 hour)


Angband Tools
User
Start/Resume Session
Save Game
View Play-Time (Death Screen)
View Play-Time (Stats Screen)
Read Timer From Savefile
Get Current Time
Calculate Session Duration
Update Total Play-Time
Write Timer To Savefile
Select Monster
Edit Monster Fields
Save Monster File
Search Monsters
Parse monster.txt
Validate Edits
Apply Edits to Monster Block
Write Updated monster.txt
<<included>>
<<include>>
<<included>>
<<include>>



#CHANGELOG.md

This changelog summarizes how Part 1 design documents were updated to match the final finished product of the Angband Monster File Editor and the play-time/timer feature.

1) Monster representation:
Original - Monsters were modeled as a set of fields and edits were put in a in-memory structure.
Final Implementation - Monster is identified as a @dataclass with better fields and categories such as (about/stats/etc..).
Reasoning - A dataclass makes the model easier to digest, avoids future bugs if there's missing attributes, and makes parsing validation easier.

2) Parsing Responsibility to a Parser Module:
Original - Parsing was inside a editor class.
Final Implementation - Parsing was split into a dedicated module. The module had helpers that helped with conversion logic and a parse monster txt function.
Reasoning - Separating the parser and the UI editor helped the code be maintained easier.

3) Editing Approach:
Original - Our writing edits could edit the whole monster file, potentially causing corruption.
Final Implementation - We instead loaded the raw lines of monster txt, we scanned for the monster name, applied edits by adding/replacing specific lines of code, then write the updates.

Reasoning - Avoids unnecessary harm and lowers the chance of breaking the file structure.

4) Repository added for search and retrieval:
Original - Search and getting happened directly in the internal list/dictionary.
Final Implementation - We used a lightweight repository that helped load monsters, support searches by name, helped retrieve the exact information we're looking for.
Reasoning - Centralized the strategy of looking up logic and keeps the editor/UI simple.

5) Mapping corrections made during development:
Original - Some keys were represented as hitPoints, armorclass, flagOff which kept the design simple.
Final Implementation - The storage and parsing is now in sync with the monster txt keys
Reasoning - Keeps edits consistent with source data and prevents parsing errors.

6) Edits with validation beforehand
Original - We assumed user edits were valid.
Final Implementation - Basic bound and other validation techniques were implemented and fails if the check did not successfully go through.
Reasoning - Prevents breaking and parsing errors from invalid values.

7) Use Case diagram updated
Original - Use cases were shown as direct actions.
Final Implementation - Updated Use Case diagram so the system responsibilities respond to user actions instead of being automated
Reasoning - The user will start the flow of the game and it simplifies the boundaries.

Timer Feature
- The timer feature is separate from the MFE feature.
- The final design documents shows which modules store the play-time fields, which write/read them in the savefile, and which UI screens display them.
- Reason: Keeps gameplay timing logic independent from the monster editor logic.